

Core Language Working Group "tentatively ready" Issues for the July, 2019 (Cologne) meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4810](#).

682. Missing description of lookup of template aliases

Section: 6.4.5 [basic.lookup.classref] **Status:** tentatively ready **Submitter:** Daveed Vandevoorde **Date:** 1 March, 2008

6.4.5 [basic.lookup.classref] does not mention template aliases as the possible result of the lookup but should do so.

Proposed resolution, June, 2019:

Change 6.4.5 [basic.lookup.classref] paragraph 1 as follows:

In a class member access expression (7.6.1.4 [expr.ref]), if the `.` or `->` token is immediately followed by an *identifier* followed by a `<`, the identifier must be looked up to determine whether the `<` is the beginning of a template argument list (13.2 [temp.names]) or a less-than operator. The identifier is first looked up in the class of the object expression (11.7 [class.member.lookup]). If the identifier is not found, it is then looked up in the context of the entire *postfix-expression* and shall name a **class** template **whose specializations are types**.

2207. Alignment of allocation function return value

Section: 6.6.5.4.1 [basic.stc.dynamic.allocation] **Status:** tentatively ready **Submitter:** Richard Smith
Date: 2015-12-02

According to 6.6.5.4.1 [basic.stc.dynamic.allocation] paragraph 2,

For an allocation function other than a reserved placement allocation function (17.6.2.3 [new.delete.placement]), the pointer returned is suitably aligned so that it can be converted to a pointer to any suitable complete object type (17.6.2.1 [new.delete.single]) and then used to access the object or array in the storage allocated (until the storage is explicitly deallocated by a call to a corresponding deallocation function).

This requirement seems excessive, as it appears to require the strictest fundamental alignment even for objects that are too small to require such an alignment.

Proposed resolution (March, 2019):

1. Change 6.6.5.4.1 [basic.stc.dynamic.allocation] paragraph 2 as follows:

...The order, contiguity, and initial value of storage allocated by successive calls to an allocation function are unspecified. ~~For an allocation function other than a reserved placement allocation function (17.6.2.3 [new.delete.placement]), the pointer returned is suitably aligned so that it can be converted to a pointer to any suitable complete object type (17.6.2.1 [new.delete.single]) and then used to access the object or array in the storage allocated (until the storage is explicitly deallocated by a call to a corresponding deallocation function).~~ Even if the size of the space requested...

2. Add the following as a new paragraph to 6.6.5.4.1 [basic.stc.dynamic.allocation] after the existing paragraph 2:

For an allocation function other than a reserved placement allocation function (17.6.2.3 [new.delete.placement]), the pointer returned on a successful call shall represent the address of storage that is aligned as follows:

- **If the allocation function takes an argument of type `std::align_val_t`, then the storage will have the alignment specified by the value of this argument.**
- **Otherwise, if the allocation function is named operator `new[]`, then the storage is aligned for any object that does not have new-extended alignment (6.6.6 [basic.align]) and is no larger than the requested size.**
- **Otherwise, the storage is aligned for any object that does not have new-extended alignment and is of the requested size.**

3. Change 17.6.2.1 [new.delete.single] paragraph 1 as follows:

Effects: The allocation functions (6.6.5.4.1 [basic.stc.dynamic.allocation]) called by a *new-expression* (7.6.2.5 [expr.new]) to allocate size bytes of storage. The second form is called for a type with new-extended alignment, and ~~allocates storage with the specified alignment. The~~ **the** first form is called otherwise, ~~and allocates storage suitably aligned to represent any object of that size provided the object's type does not have new-extended alignment.~~

4. Change 17.6.2.2 [new.delete.array] paragraph 1 as follows:

Effects: The allocation functions (6.6.5.4.1 [basic.stc.dynamic.allocation]) called by the array form of a *new-expression* (7.6.2.5 [expr.new]) to allocate size bytes of storage. The second form is called for a type with new-extended alignment, and ~~allocates storage with the specified alignment. The~~ **the** first form is called otherwise, ~~and allocates storage suitably aligned to represent any array object of that size or smaller, provided the object's type does not have new-extended alignment.~~²¹⁸

2300. Lambdas in multiple definitions

A lambda expression in two translation units has distinct closure types, because each such expression's type is unique within the program. This results in an issue with the ODR, which requires that the definitions of an entity are identical. For example, if

```
template <int> void f() {std::function<void()> f = []{};}
```

appears in two translation units, different specializations of function's constructor template are called, which violates 6.2 [basic.def.odr] bullet 6.4.

Issue 765 dealt with a similar problem for inline functions, but the issue still remains for templates.

Proposed resolution, April, 2019:

Change 6.2 [basic.def.odr] paragraph 12 as follows:

...Given such an entity named *D* defined in more than one translation unit, then

- each definition of *D* shall consist of the same sequence of tokens, **for which the definition of a closure type is considered to consist of the sequence of tokens of the corresponding lambda-expression**; and
- in each definition of *D*, corresponding names, looked up according to 6.4 [basic.lookup], ~~shall refer to an entity defined within the definition of *D*, or~~ shall refer to the same entity, after overload resolution (12.3 [over.match]) and after matching of partial template specialization (13.9.3 [temp.over]), except that a name can refer to

- a non-volatile const object with internal or no linkage if the object

- has the same literal type in all definitions of *D*,
- is initialized with a constant expression (7.7 [expr.const]),
- is not odr-used in any definition of *D*, and
- has the same value in all definitions of *D*,

or

- a reference with internal or no linkage initialized with a constant expression such that the reference refers to the same entity in all definitions of *D*;

and

- **in each definition of *D*, except within the default arguments and default template arguments of *D*, corresponding lambda-expressions shall have the same closure type (see below), and**
- in each definition of *D*, corresponding entities shall have the same language linkage; and
- in each definition of *D*, the overloaded operators referred to, the implicit calls to conversion functions, constructors, operator new functions and operator delete functions, shall refer to the same function, ~~or to a function defined within the definition of *D*~~; and
- in each definition of *D*, a default argument used by an (implicit or explicit) function call **or a default template argument used by an (implicit or explicit) template-id or simple-template-id** is treated as if its token sequence were present in the definition of *D*; that is, the default argument **or default template argument** is subject to the requirements described in this

paragraph (and, if the default argument has subexpressions with default arguments, this requirement applies recursively) [Footnote: 9.2.3.6 [del.fct.default] describes how default argument names are looked up. —end footnote]; and

- ...

If *D* is a template and is defined in more than one translation unit, then the preceding requirements shall apply both to names from the template's enclosing scope used in the template definition (13.7.3 [temp.nondep]), and also to dependent names at the point of instantiation (13.7.2 [temp.dep]). If the definitions of *D* satisfy all these requirements, then the behavior is as if there were a single definition of *D*. These requirements also apply to corresponding entities defined within each definition of *D* (including the closure types of *lambda-expressions*, but excluding entities defined within default arguments or default template arguments of either *D* or an entity not defined within *D*). For each such entity and for *D* itself, the behavior is as if there is a single entity with a single definition, including in the application of these requirements to other entities. [Note: The entity is still declared in multiple translation units, and 6.5 [basic.link] still applies to these declarations. In particular, *lambda-expressions* (7.5.5 [expr.prim.lambda]) appearing in the type of *D* may result in the different declarations having distinct types, and *lambda-expressions* appearing in a default argument of *D* may still denote different types in different translation units. —end note] If the definitions of *D* do not satisfy these requirements, then the behavior is undefined. program is ill-formed, no diagnostic required. [Example:

```
inline void f(bool cond, void (*p)()) {
    if (cond) f(false, []{});
}
inline void g(bool cond, void (*p)() = []{}) {
    if (cond) g(false);
}
struct X {
    void h(bool cond, void (*p)() = []{}) {
        if (cond) h(false);
    }
}
```

If the definition of *f* appears in multiple translation units, the behavior of the program is as if there is only one definition of *f*. If the definition of *g* appears in multiple translation units, the program is ill-formed (no diagnostic required) because each such definition uses a default argument that refers to a distinct *lambda-expression* closure type. The definition of *x* can appear in multiple translation units of a valid program; the *lambda-expressions* defined within the default argument of *x*: *h* within the definition of *x* denote the same closure type in each translation unit. —end example]

2366. Can default initialization be constant initialization?

Section: 6.8.3.2 [basic.start.static] **Status:** tentatively ready **Submitter:** Geoffrey Romer **Date:** 2017-11-01

According to 6.8.3.2 [basic.start.static] paragraph 2,

Constant initialization is performed if a variable or temporary object with static or thread storage duration is initialized by a constant initializer for the entity. If constant initialization is not performed, a variable with static storage duration (6.6.5.1 [basic.stc.static]) or thread storage duration (6.6.5.2

[basic.stc.thread]) is zero-initialized (9.3 [dcl.init]). Together, zero-initialization and constant initialization are called static initialization; all other initialization is *dynamic initialization*.

This appears to require an explicit initializer for constant initialization and would preclude examples like:

```
struct S {  
    int i = 1;  
};  
static constexpr S s;
```

where there is no initializer.

Notes from the October, 2018 teleconference:

CWG agreed that the example should be well-formed.

Proposed resolution, June, 2019:

1. Change 6.8.3.2 [basic.start.static] paragraph 2 as follows:

Constant initialization is performed if a variable or temporary object with static or thread storage duration is **constant-initialized** by a constant initializer (7.7 [expr.const]) for the entity. If constant initialization is not performed, a variable with static storage duration (6.6.5.1 [basic.stc.static]) or thread storage duration (6.6.5.2 [basic.stc.thread]) is zero-initialized (9.3 [dcl.init]). Together, zero-initialization and constant initialization...

2. Change 7.7 [expr.const] paragraph 2 as follows:

A **constant initializer** for a variable or temporary object *o* is an initializer for which interpreting its full-expression as a constant-expression results in a constant expression **constant-initialized** if

- **either it has an initializer or its default-initialization results in some initialization being performed, and**
- **its initialization full-expression is a constant expression when interpreted as a constant-expression**, except that if *o* is an object, **such an initializer the initialization full-expression** may also invoke constexpr constructors for *o* and its subobjects even if those objects are of non-literal class types. [Note: Such a class may have a non-trivial destructor. Within this evaluation, `std::is_constant_evaluated()` (20.15.10 [meta.const.eval]) returns true. —end note]

3. Change 7.7 [expr.const] paragraph 3 as follows:

A variable is *usable in constant expressions* after its initializing declaration is encountered if it is a constexpr variable, or it is **a constant-initialized variable** of reference type or of const-qualified integral or enumeration type, **and its initializer is a constant initializer**.

4. Change 9.1.5 [dcl.constexpr] paragraph 5 as follows:

For a constexpr function or constexpr constructor that is neither defaulted nor a template, if no argument values exist such that an invocation of the function or constructor could be an evaluated subexpression of a core constant expression (7.7 [expr.const]), or, for a constructor, **a constant initializer for an evaluated subexpression of the initialization full-expression of** some **constant-initialized** object (6.8.3.2 [basic.start.static]), the program is ill-formed, no diagnostic required. [Example:...

2376. Class template argument deduction with array declarator

Section: 12.3.1.8 [over.match.class.deduct] **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 2018-03-01

An example like

```
template <class ...T> struct A {  
    A(T...) {}  
};  
A x[29]{};
```

Appears to be permitted by the current wording of the Standard, but existing implementations reject it. Should this usage be supported (in which case some mention of it in the wording would be useful) or prohibited?

Notes from the November, 2018 meeting:

The example is intended to be ill-formed; the intent is that declarator operators are not permitted, as with `decltype(auto)`.

Proposed resolution, March, 2019:

Change 9.1.7.6 [dcl.type.class.deduct] paragraph 1 as follows:

If a placeholder for a deduced class type appears as a *decl-specifier* in the *decl-specifier-seq* of an initializing declaration (9.3 [dcl.init]) of a variable, the **declared type of the variable shall be cv τ , where τ is the placeholder.** [Example:

```
template <class ...T> struct A {  
    A(T...) {}  
};  
A x[29]{}; // error: no declarator operators allowed  
const A& y{}; // error: no declarator operators allowed
```

—end example] The placeholder is replaced by the return type of the function selected by overload resolution for class template deduction (12.3.1.8 [over.match.class.deduct]). If the *decl-specifier-seq* is followed by an *init-declarator-list* or *member-declarator-list* containing more than one declarator, the type that replaces the placeholder shall be the same in each deduction.

2390. Is the argument of `__has_cpp_attribute` macro-expanded?

Section: 15.1 [cpp.cond] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2018-11-14

The Standard does not specify whether the argument of `__has_cpp_attribute` is macro-expanded before being tested to see if it names an attribute or not, and there is implementation divergence.

Notes from the February, 2019 meeting:

CWG observed that a use of an attribute would be macro-expanded, so it seemed reasonable to expect to be able to specify the same macro as the argument to `__has_cpp_attribute` and get the corresponding result.

Proposed resolution (June, 2019):

1. Change 15.1 [cpp.cond] paragraph 5 as follows:

Each *has-attribute-expression* is replaced by a non-zero *pp-number* matching the form of an *integer-literal* if the implementation supports an attribute with the name specified by interpreting the *pp-tokens*, **after macro expansion**, as an *attribute-token*, and by 0 otherwise. The program is ill-formed if the *pp-tokens* do not match the form of an *attribute-token*.

2. Change 15.1 [cpp.cond] paragraph 11 as follows:

After all replacements due to macro expansion and evaluations of *defined-macro-expressions*, **and** *has-include-expressions*, **and** *has_attribute_expressions* have been performed, all remaining identifiers and keywords, except for `true` and `false`, are replaced with the *pp-number* 0, and then each preprocessing token is converted into a token. [Note: An alternative token (5.5 [lex.digraph]) is not an identifier, even when its spelling consists entirely of letters and underscores. Therefore it is not subject to this replacement. —end note]

2400. Constexpr virtual functions and temporary objects

Section: 7.7 [expr.const] **Status:** tentatively ready **Submitter:** Daveed Vandevoorde **Date:** 2019-02-08

Consider an example like the following:

```
struct A {
    constexpr virtual int f() const {
        return 1;
    }
};

struct B : A {
    constexpr virtual int f() const {
        return 2;
    }
};

constexpr B b{};
constexpr A&& ref = (B)b;

static_assert(ref.f() == 2, "");
```

Since the temporary bound to `ref` is non-const, it can be re-newed to something else, which would make the invocation `ref.f()` undefined behavior, which the interpreter is required to catch.

Presumably, `ref.f()` should not be a constant expression, and 7.7 [expr.const] paragraph 2 should have a bullet for invoking a virtual function of a non-const object unless its lifetime began within the evaluation of the constant expression.

Proposed resolution (March, 2019):

Add the following as a new bullet following 7.7 [expr.const] bullet 4.4:

An expression *e* is a *core constant expression* unless the evaluation of *e*, following the rules of the abstract machine (6.8.1 [intro.execution]), would evaluate one of the following expressions:

- ...
- an invocation of an instantiated constexpr function or constexpr constructor that fails to satisfy the requirements for a constexpr function or constexpr constructor (9.1.5 [dcl.constexpr]);
- **an invocation of a virtual function (11.6.2 [class.virtual]) for an object unless**
 - **the object is usable in constant expressions or**
 - **its lifetime began within the evaluation of *e*;**
- an expression that would exceed the implementation-defined limits (see Annex B [implimits]);
- ...

2404. [[no_unique_address]] and allocation order

Section: 11.3 [class.mem] **Status:** tentatively ready **Submitter:** Daveed Vandevoorde **Date:** 2018-12-13

According to 11.3 [class.mem] paragraph 19,

Non-static data members of a (non-union) class with the same access control (11.8 [class.access]) are allocated so that later members have higher addresses within a class object.

With the advent of the [[no_unique_address]] attribute, “higher addresses” is no longer strictly accurate. According to the FAQ in [P0840R2](#), next-to-last question:

Q: Suppose I have members *a*, *b*, *c* (in that order, with the same access). Today we guarantee that $\&a < \&b < \&c$. What happens if *b* has the attribute?

Two cases:

1. If the type of *b* is empty, then there is no guarantee about the address of *b* (other than that it is somewhere within the containing object).
2. If the type of *b* is nonempty, then we still guarantee that $\&a < \&b < \&c$.

Presumably the wording in 11.3 [class.mem] paragraph 19 needs to be changed to reflect that intent.

Proposed resolution, March, 2019:

Change 11.3 [class.mem] paragraph 19 as follows:

[Note: Non-static data members of a (non-union) class with the same access control (11.8 [class.access]) **and non-zero size (6.6.2 [intro.object])** are allocated so that later members have higher addresses within a class object. The order of allocation of non-static data members with different access control is unspecified (~~11.8 [class.access]~~). Implementation alignment requirements might cause two adjacent members not to be allocated immediately after each other; so might requirements for space for managing virtual functions (11.6.2 [class.virtual]) and virtual base classes (11.6.1 [class.mi]). **—end note]**

2406. `[[fallthrough]]` attribute and iteration statements

Section: 9.11.6 `[dcl.attr.fallthrough]` **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 2019-03-08

According to 9.11.6 `[dcl.attr.fallthrough]` paragraph 1,

A `fallthrough` statement may only appear within an enclosing `switch` statement (8.4.2 `[stmt.switch]`). The next statement that would be executed after a `fallthrough` statement shall be a labeled statement whose label is a case label or default label for the same `switch` statement.

The meaning of “next statement that would be executed” is unclear with respect to the controlled substatement of an iteration statement. There is implementation divergence on an example like:

```
void f(int n) {
    switch (n) {
        case 0:
            while (true)
                [[fallthrough]]; // Well-formed?
        case 1:
            break;
    }
}
```

Proposed resolution, April, 2019:

1. Change 9.11.6 `[dcl.attr.fallthrough]` paragraph 1 as follows:

The *attribute-token* `fallthrough` may be applied to a null statement (8.2 `[stmt.expr]`); such a statement is a `fallthrough` statement. The *attribute-token* `fallthrough` shall appear at most once in each *attribute-list* and no *attribute-argument-clause* shall be present. A `fallthrough` statement may only appear within an enclosing `switch` statement (8.4.2 `[stmt.switch]`). The next statement that would be executed after a `fallthrough` statement shall be a labeled statement whose label is a case label or default label for the same `switch` statement **and, if the `fallthrough` statement is contained in an iteration statement, the next statement shall be part of the same execution of the substatement of the innermost enclosing iteration statement**. The program is ill-formed if there is no such statement.

2. Change the example in 9.11.6 `[dcl.attr.fallthrough]` paragraph 3 as follows:

[Example:

```
void f(int n) {
    void g(), h(), i();
    switch (n) {
        case 1:
        case 2:
            g();
            [[fallthrough]];
        case 3: // warning on fallthrough discouraged
            do {
                [[fallthrough]]; // error: next statement is not part of the same substatement execution
            } while (false);
        case 6:
            do {
                [[fallthrough]]; // error: next statement is not part of the same substatement execution
```

```

    } while (n--);
    case 7:
        while (false) {
            [[fallthrough]]; // error: next statement is not part of the same substatement execution
        }
    case 5:
        h();
    case 4:                // implementation may warn on fallthrough
        i();
        [[fallthrough]]; // ill-formed
    }
}
—end example]

```

2418. Missing cases in definition of “usable in constant expressions”

Section: 7.7 [expr.const] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2018-11-25

The term “usable in constant expressions” (7.7 [expr.const] paragraph 3) is only defined for variables:

A variable is *usable in constant expressions* after its initializing declaration is encountered if it is a constexpr variable, or it is of reference type or of const-qualified integral or enumeration type, and its initializer is a constant initializer.

However, uses of the term assume that it applies more widely. For example, 7.7 [expr.const] bullet 4.7.1 mentions “a non-volatile glvalue that refers to an object that is usable in constant expressions” (not all objects are variables), and bullet 4.10.1 speaks of a “data member of reference type” (also not a variable) that is usable in constant expressions.

Proposed resolution, June, 2019:

Change 7.7 [expr.const] paragraph 3 as follows:

A variable is usable in constant expressions after its initializing declaration is encountered if it is a constexpr variable, or it is of reference type or of const-qualified integral or enumeration type, and its initializer is a constant initializer. **An object or reference is usable in constant expressions if it is**

- **a variable that is usable in constant expressions, or**
- **a template parameter object (13.1 [temp.param]), or**
- **a string literal object (5.13.5 [lex.string]), or**
- **a non-mutable subobject or reference member of any of the above, or**
- **a complete temporary object of non-volatile const-qualified integral or enumeration type that is initialized with a constant expression.**